CS224N Assignment 4: Written Solutions

Diego Alcaraz

1. Attention Exploration (Written)

We are given (single-head) attention:

$$c = \sum_{i=1}^{n} v_i \alpha_i, \qquad \alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^{n} \exp(k_j^\top q)}.$$

The key idea is: *the softmax makes the largest dot product dominate.* So if one $k_j^\top q$ is much larger than all others, $\alpha_j \approx 1$ and the output c basically *copies* v_j.

(a) Copying in attention

(i)

The categorical distribution α puts almost all its mass on some index j when

$$k_j^\top q \gg k_i^\top q \quad \text{for all } i \neq j.$$

Equivalently, $\exp(k_j^\top q)$ is overwhelmingly larger than the sum of all the other $\exp(k_i^\top q)$ terms, which happens when q is strongly aligned with k_j (large cosine similarity) and/or has large magnitude in that aligned direction.

(ii)

Under that condition, $\alpha_j \approx 1$ and $\alpha_i \approx 0$ for $i \neq j$, so

$$c = \sum_{i=1}^{n} v_i \alpha_i \approx v_j.$$

So attention can **copy** a specific value vector into the output.

(b) An average of two

We want $c \approx \frac{1}{2}(v_a + v_b)$.

Assumptions: all keys are orthogonal ($k_i^\top k_j = 0$ if $i \neq j$) and all keys have norm 1.

A clean way is to make the logits equal for a and b, and much larger than the rest. If we choose

$$q = \beta(k_a + k_b) \quad \text{with } \beta \gg 0,$$

then:

$$k_a^\top q = \beta(k_a^\top k_a + k_a^\top k_b) = \beta(1 + 0) = \beta,$$

$$k_b^\top q = \beta(k_b^\top k_a + k_b^\top k_b) = \beta(0+1) = \beta,$$

and for any $i \neq a, b$,

$$k_i^\top q = \beta(k_i^\top k_a + k_i^\top k_b) = \beta(0+0) = 0.$$

Thus the attention weights are:

$$\alpha_a = \frac{e^\beta}{e^\beta + e^\beta + \sum_{i \neq a,b} e^0} = \frac{e^\beta}{2e^\beta + (n-2)}, \quad \alpha_b = \frac{e^\beta}{2e^\beta + (n-2)}.$$

If β is large, then $2e^\beta \gg (n-2)$, so:

$$\alpha_a \approx \alpha_b \approx \frac{1}{2}, \qquad \alpha_i \approx 0 \ (i \neq a, b),$$

and therefore

$$c = \sum_i v_i \alpha_i \approx \tfrac{1}{2} v_a + \tfrac{1}{2} v_b = \tfrac{1}{2}(v_a + v_b).$$

(c) Drawbacks of single-headed attention

The point: yes, we can make single-head attention approximate an average over a chosen set, but to do that we must engineer q so that the dot products for the chosen items are equal and large, while all others are small. In practice:

[leftmargin=18pt]

- The model does not get to choose the subset "for free"; it must encode this combinatorial selection inside q.
- If keys are noisy / not perfectly orthogonal / norms vary, softmax can become unstable and one item can dominate.
- One head gives only one distribution α, so it cannot robustly represent multiple different "views" (e.g., syntax + coreference) at the same time.

So single-head can do it *in principle*, but it is brittle and hard to generalize.

(c.i) Random keys with small covariance: construct q from means

Setup (as in the handout): keys are sampled $k_i \sim \mathcal{N}(\mu_i, \Sigma_i)$, means μ_i are mutually orthogonal and $\|\mu_i\| = 1$. Assume $\Sigma_i = \alpha I$ for vanishingly small α.

We want $c \approx \frac{1}{2}(v_a + v_b)$ *in expectation / typically*, even though we only know means. Since $k_i \approx \mu_i$ (noise is tiny), we can use the same idea as in (b) but replacing unknown k by known μ:

$$q = \beta(\mu_a + \mu_b), \quad \beta \gg 0.$$

Then typically $k_a^\top q \approx \mu_a^\top \beta(\mu_a + \mu_b) = \beta$ and $k_b^\top q \approx \beta$, while $k_i^\top q \approx 0$ for $i \neq a, b$. Hence $\alpha_a \approx \alpha_b \approx \frac{1}{2}$ and $c \approx \frac{1}{2}(v_a + v_b)$.

(c.ii) Large-norm perturbation for one key: qualitative behavior of c (using older-semester idea)

Now $\Sigma_a = \alpha I + \frac{1}{2}(\mu_a \mu_a^\top)$ (with tiny α), so k_a keeps pointing near μ_a but its *magnitude* varies a lot. For $i \neq a$, $\Sigma_i = \alpha I$, so $k_i \approx \mu_i$.

Using the query from part (c.i), take:

$$q = \beta(\mu_a + \mu_b), \quad \beta \gg 0.$$

Because the special covariance makes the component along μ_a vary, we can think (qualitatively) of:

$$k_a \approx \gamma \mu_a, \quad \gamma \text{ varies across samples}, \qquad k_b \approx \mu_b.$$

Then the two important logits are:

$$k_a^\top q \approx (\gamma \mu_a)^\top \beta(\mu_a + \mu_b) = \gamma\beta, \qquad k_b^\top q \approx \mu_b^\top \beta(\mu_a + \mu_b) = \beta.$$

So the weights become approximately:

$$\alpha_a \approx \frac{e^{\gamma\beta}}{e^{\gamma\beta} + e^{\beta}}, \qquad \alpha_b \approx \frac{e^{\beta}}{e^{\gamma\beta} + e^{\beta}}.$$

If $\gamma > 1$, then $\gamma\beta > \beta$ and α_a dominates, so $c \approx v_a$. If $\gamma < 1$, then α_b dominates, so $c \approx v_b$.

Conclusion: across different samples of k_a, the output c can *flip* between being close to v_a or close to v_b, instead of stably averaging them. So the variance of c is much larger than in part (c.i).

(d) Benefits of multi-headed attention

Here we have two heads with queries q_1, q_2, producing c_1, c_2, and the final output is:

$$c = \tfrac{1}{2}(c_1 + c_2).$$

The big win is: **each head can specialize**, and averaging outputs can reduce brittleness.

(d.i) $\Sigma_i = \alpha I$ small: choose distinct q_1, q_2

We want the final $c \approx \frac{1}{2}(v_a + v_b)$, but with *different expressions* for q_1, q_2.

A simple robust approach:

$$q_1 = \beta\mu_a, \qquad q_2 = \beta\mu_b, \qquad \beta \gg 0.$$

Then head 1 concentrates on item a:

$$k_a^\top q_1 \approx \mu_a^\top \beta \mu_a = \beta, \quad k_i^\top q_1 \approx 0 \ (i \neq a), \Rightarrow c_1 \approx v_a.$$

Similarly head 2 concentrates on item b, giving $c_2 \approx v_b$.

Therefore

$$c = \tfrac{1}{2}(c_1 + c_2) \approx \tfrac{1}{2}(v_a + v_b).$$

This avoids needing one head to do a perfectly balanced softmax between two items.

(d.ii) When one key has unstable norm, why multi-head helps (intuition)

If k_a has unstable magnitude, then any *single* softmax that tries to balance a and b can be hijacked by the random norm (as in (c.ii)). With two heads, we can:

[leftmargin=18pt]

- Use one head that targets a (it will still sometimes be strong/weak, but it is *supposed* to focus on a).
- Use another head that targets b, independently.

Averaging c_1 and c_2 stabilizes the combined result: even if head 1 occasionally over/under-focuses due to norm noise, head 2 still reliably contributes v_b.

3. RoPE (Rotary Positional Embedding) — Written

(i) Show Eq. (3) elements can be obtained from Eq. (4)

RoPE applies a 2D rotation to each pair of coordinates:

$$\begin{pmatrix} x_t^{(2i-1)} \\ x_t^{(2i)} \end{pmatrix} \mapsto \begin{pmatrix} \cos(t\theta_i) & -\sin(t\theta_i) \\ \sin(t\theta_i) & \cos(t\theta_i) \end{pmatrix} \begin{pmatrix} x_t^{(2i-1)} \\ x_t^{(2i)} \end{pmatrix}.$$

Now represent each pair as a complex number:

$$z_i \;=\; x_t^{(2i-1)} + i\, x_t^{(2i)}.$$

Multiplying by $e^{it\theta_i} = \cos(t\theta_i) + i\sin(t\theta_i)$ gives:

$$e^{it\theta_i} z_i = (\cos(t\theta_i) + i\sin(t\theta_i))(x_t^{(2i-1)} + ix_t^{(2i)}).$$

Expanding:

$$= \big(\cos(t\theta_i)x_t^{(2i-1)} - \sin(t\theta_i)x_t^{(2i)}\big) \;+\; i\big(\sin(t\theta_i)x_t^{(2i-1)} + \cos(t\theta_i)x_t^{(2i)}\big).$$

So the **real part** is the new $(2i-1)$-th coordinate and the **imaginary part** is the new $(2i)$-th coordinate, which matches exactly the block-rotation in Eq. (3).

Eq. (4) simply stacks these $e^{it\theta_i}$ as a vector and multiplies them elementwise with the stacked complex pairs z_i. After that, you reshape back:

$$\big[\Re(\cdot), \Im(\cdot)\big] \;\Rightarrow\; \mathbb{R}^d,$$

which reproduces the rotated real vector in Eq. (3).

(ii) Relative embeddings dot product depends only on $t_1 - t_2$ (2D case)

Let $z_1, z_2 \in \mathbb{C}$ represent 2D real vectors. RoPE is:

$$\text{RoPE}(z, t) = e^{it\theta} z.$$

Dot product in the prompt: $\langle z_1, z_2 \rangle = \Re(z_1 \overline{z_2})$.

Then:

$$\langle \text{RoPE}(z_1, t_1), \text{RoPE}(z_2, t_2) \rangle = \Re\Big((e^{it_1\theta} z_1)\overline{(e^{it_2\theta} z_2)} \Big) = \Re\Big(e^{it_1\theta} z_1 \cdot e^{-it_2\theta} \overline{z_2} \Big) = \Re\Big(e^{i(t_1 - t_2)\theta} z_1 \overline{z_2} \Big).$$

But this is exactly:

$$\Re\Big((e^{i(t_1 - t_2)\theta} z_1)\overline{(e^{i \cdot 0 \cdot \theta} z_2)} \Big) = \langle \text{RoPE}(z_1, t_1 - t_2), \text{RoPE}(z_2, 0) \rangle.$$

So the dot product depends on the relative position $t_1 - t_2$ only.

Question 4: Considerations in Pretrained Knowledge (5 points)

Please type the answers to these written questions (to make TA lives easier).

[label=()]

1. **(1 point)** Succinctly explain why the pretrained (vanilla) model was able to achieve an accuracy of above 10%, whereas the non-pretrained model was not.

   Answer: The pretrained (vanilla) model has already learned general language patterns from a much larger corpus, so it can recognize relationships between words and typical ways facts appear in text. In contrast, the non-pretrained model starts from random weights and only sees the small task dataset, so it cannot build strong representations or world-knowledge-like associations within the limited training signal. Because pretraining gives the model a strong prior over how language works, fine-tuning only needs to adapt that prior to the specific task format.

2. **(2 points)** Take a look at some of the correct predictions of the pretrain+finetuned vanilla model, as well as some of the errors. We think you'll find that it's impossible to tell, just looking at the output, whether the model retrieved the correct birth place, or made up an incorrect birth place. Consider the implications for user-facing systems that involve pretrained NLP components. Come up with two **distinct** reasons why this model behavior (i.e., unable to tell whether it's retrieved or made up) may cause concern for such applications, and an example for each reason.

   Answer:

   [label=0.]

   (a) Such behavior may cause users to rely on false information in their work unintentionally. For instance, if a user wants to mention the birthplace of a famous person, they might cite the wrong place if the model outputs it confidently without any indicator of uncertainty. This could reduce the quality of reports, essays, or decisions based on the answer.

   (b) Such behavior may cause misinformation to spread socially because the output *looks* like a confident factual statement. For example, if the model outputs an incorrect birthplace for a public figure, a user might share it with others (or post it online), and the claim can propagate as a "fact". This can create confusion, harm trust, and make it harder to correct errors later.

3. **(2 points)** If your model didn't see a person's name at pretraining time, and that person was not seen at fine-tuning time either, it is not possible for it to have "learned" where they lived. Yet, your model will produce *something* as a predicted birth place for that person's name if asked. Concisely describe a strategy your model might take for predicting a birth place for that person's name, and one reason why this should cause concern for the use of such applications. (You do not need to submit the same answer for 4c as for 4b.)

   Answer: The model may use a heuristic strategy: given an unfamiliar name, it tries to map it to something *similar* in its parameters (e.g., names with similar spelling, language origin, or co-occurring context patterns), and then outputs a plausible birthplace that often appears with those similar names. This is concerning because the prediction is not grounded in any real evidence about that person; it is essentially a best-guess completion that can sound authoritative, which can mislead users into believing the output is a verified fact.